JOUR 1

Fondements du Deep Learning et géométrie des représentations neuronales

27 juillet 2026

Intervenants et thèmes principaux :

Session 1 — Des modèles linéaires aux réseaux de neurones profonds

Session 2 — Géométrie des représentations (hypothèse platonicienne)

Avant-propos

Ce document constitue une restitution structurée de la première journée de l'Eastern European Machine Learning Summer School (EEML) 2026, suivie à distance. Il ne s'agit pas d'une transcription brute de l'intervention orale, mais d'une réorganisation pédagogique du contenu : les notions sont regroupées par thème, reformulées dans un registre académique neutre et présentées selon une progression logique plutôt que selon l'ordre chronologique du direct.

La journée a comporté deux interventions principales. La première portait sur les fondements des réseaux de neurones profonds, depuis les limites des modèles linéaires jusqu'aux techniques ayant permis l'essor du deep learning moderne. La seconde, consacrée à la géométrie des représentations neuronales, a exploré la question de savoir dans quelle mesure différents modèles d'intelligence artificielle apprennent des représentations comparables du monde.

Le corps du document présente, pour chaque chapitre, une synthèse concise des idées essentielles. Les développements plus détaillés — démonstrations intuitives, exemples numériques, schémas commentés — sont regroupés en annexe, avec un renvoi explicite depuis chaque chapitre concerné, conformément à la structure demandée.

Partie I — Des modèles linéaires aux réseaux de neurones profonds

Chapitre 1. Les limites des modèles linéaires : le problème XOR

La régression logistique et le perceptron sont deux modèles d'apprentissage très proches : tous deux définissent une frontière de décision linéaire de la forme $w_1x_1 + w_2x_2 + b = 0$. Cette frontière ne fonctionne que lorsque les classes sont linéairement séparables.

Idées clés

- En 1969, Marvin Minsky et Seymour Papert démontrent, dans leur ouvrage Perceptrons, qu'un perceptron à une seule couche ne peut pas apprendre la fonction logique XOR (« ou exclusif »), car ce problème n'est pas linéairement séparable.
- Cette démonstration a été généralisée à tort : elle a conduit une partie de la communauté scientifique à conclure que les réseaux de neurones, plus largement, étaient d'une capacité limitée.
- Les conséquences ont été notables : réduction des financements et ralentissement des recherches sur les réseaux de neurones — une période aujourd'hui connue sous le nom de premier « hiver de l'IA ».
- La solution — l'ajout d'une couche cachée — était pourtant déjà à portée : un réseau multicouche permet de construire des frontières de décision non linéaires et résout le problème XOR sans difficulté.

Détails, table de vérité et représentation graphique : voir Annexe A.1.

Chapitre 2. Du feature engineering à l'apprentissage de représentations

Face à un problème non linéairement séparable, deux stratégies sont possibles : changer de modèle, ou changer la représentation des données. La seconde approche consiste à transformer les variables d'entrée x en de nouvelles variables $\phi(x)$, choisies de sorte que le problème redevienne linéairement séparable. Ce travail de construction manuelle de variables est appelé feature engineering (ingénierie des caractéristiques).

Idées clés

- Le feature engineering a dominé la recherche en apprentissage automatique dans les années 1980-1990, notamment en vision par ordinateur.
- Cette approche devient impraticable dès que la dimensionnalité des données augmente (des milliers, voire des millions de variables dans le cas d'images).
- L'idée fondatrice du deep learning est d'inverser la logique : au lieu de fabriquer les caractéristiques à la main, on laisse le modèle les apprendre directement à partir des données. C'est le principe du representation learning (apprentissage de représentations).
- Sur le plan architectural, cela se traduit par l'ajout d'une couche cachée (hidden layer) entre l'entrée et la classification : cette couche apprend automatiquement une nouvelle représentation utile des données, sans intervention humaine directe.

Détails, schémas et exemple appliqué à la reconnaissance faciale : voir Annexe A.2.

Chapitre 3. Anatomie d'un réseau de neurones feedforward

Un réseau de neurones artificiel s'inspire du fonctionnement du cerveau biologique, sans en constituer une réplique fidèle : un seul neurone biologique reste considérablement plus complexe qu'un neurone artificiel. Cette architecture porte plusieurs noms selon la propriété que l'on souhaite souligner : réseau de neurones artificiel (ANN), réseau feedforward (l'information circule uniquement de l'entrée vers la sortie) et réseau entièrement connecté (fully connected), où chaque neurone d'une couche est relié à tous les neurones de la couche suivante.

Le calcul, couche par couche

- Pré-activation : $a(x) = W_1^T x + b_1$ — une simple combinaison linéaire des entrées.
- Activation : $h(x) = g(a(x))$, où g est une fonction non linéaire (sigmoïde, tanh…). C'est cette étape qui introduit la non-linéarité.
- Sortie : $y = f(W_2^T h + b_2)$, où f est choisie selon la tâche (identité pour la régression, sigmoïde pour la classification binaire, softmax pour la classification multiclasse).

Détails sur les fonctions d'activation et le choix de la couche de sortie : voir Annexe A.3.

Chapitre 4. Entraînement d'un réseau : descente de gradient et rétropropagation

Pourquoi une fonction non linéaire est-elle indispensable ? Parce que la composition de deux transformations linéaires reste une transformation linéaire : empiler des couches purement linéaires n'apporte donc aucune puissance de modélisation supplémentaire par rapport à un modèle linéaire simple. L'introduction d'une non-linéarité (sigmoïde, tanh, puis plus tard ReLU) est ce qui confère au réseau sa capacité à modéliser des relations complexes.

Idées clés

- L'entraînement d'un réseau de neurones repose sur les mêmes principes que ceux de la régression logistique : descente de gradient et minimisation d'une fonction de perte.
- Le calcul des gradients s'effectue par application de la règle de dérivation en chaîne, depuis la sortie jusqu'à l'entrée.
- La rétropropagation (backpropagation) est l'algorithme qui applique cette règle de manière efficace, par programmation dynamique, ce qui rend l'entraînement de réseaux comptant des millions de paramètres possible en pratique.

- Historiquement, l'algorithme a été décrit dès 1974 dans la thèse de Paul Werbos, avant d'être popularisé par l'article de David Rumelhart, Geoffrey Hinton et Ronald Williams en 1986.
- Les bibliothèques modernes (PyTorch, TensorFlow, JAX) automatisent entièrement ce calcul grâce à la différentiation automatique (autodiff), ce qui a considérablement accéléré la recherche en deep learning.

Détails et repères historiques sur les frameworks de calcul de gradients : voir Annexe A.4.

Chapitre 5. Expressivité et capacité d'apprentissage

Le théorème d'approximation universelle (Hornik, 1991) établit qu'un réseau de neurones à une seule couche cachée peut approximer n'importe quelle fonction continue, à condition de disposer d'un nombre suffisant de neurones. Ce résultat, bien que puissant en apparence, appelle une nuance essentielle.

Idées clés

- La formulation « à condition d'avoir suffisamment de neurones » cache un piège : ce nombre peut croître de façon exponentielle avec la complexité du problème, ce qui expose le modèle au surapprentissage (overfitting).
- Il faut distinguer deux notions : l'expressivité, c'est-à-dire ce qu'un modèle peut représenter en théorie, et la capacité d'apprentissage (learnability), c'est-à-dire ce que l'algorithme d'entraînement (la descente de gradient) parvient réellement à trouver.
- Un jeu de poids optimal peut exister sans qu'aucun algorithme connu ne soit capable de le découvrir.
- C'est cette limite qui motive le passage à des réseaux profonds (plusieurs couches successives) plutôt qu'à un unique réseau très large : chaque couche effectue une transformation modérée, et l'empilement de plusieurs couches permet d'atteindre une séparabilité satisfaisante sans recourir à un nombre exponentiel de neurones.

Détails et illustration progressive de la séparation des données couche après couche : voir Annexe A.5.

Chapitre 6. Le problème du gradient qui s'annule et ses remèdes

Dans les années 1990, les réseaux profonds n'ont pas tenu leurs promesses : un réseau à trois ou cinq couches obtenait parfois de moins bons résultats qu'un réseau à une seule couche cachée. La cause identifiée est le phénomène du vanishing gradient (gradient qui s'annule) : les dérivées des fonctions sigmoïde et tanh étant toujours inférieures à 1, leur multiplication successive à travers de nombreuses couches fait tendre le gradient vers zéro, empêchant l'apprentissage des couches les plus profondes.

Solutions historiques

- Le greedy layer-wise pretraining (Hinton et al., 2006 ; Bengio et al.) entraîne le réseau couche par couche à l'aide d'autoencodeurs — des réseaux apprenant, sans étiquettes, à reconstruire leur propre entrée à partir d'une représentation compressée. Chaque couche est figée avant l'entraînement de la suivante, puis l'ensemble est affiné (supervised fine-tuning) sur la tâche finale. Cette méthode, précurseur du couple pré-entraînement/fine-tuning utilisé aujourd'hui pour les grands modèles de langage, améliore l'initialisation des poids mais ne résout pas fondamentalement le problème du gradient.
- Le remplacement des fonctions d'activation constitue la solution la plus directe : la fonction ReLU (Rectified Linear Unit), f(x) = max(0, x), possède une dérivée constante (1) sur sa partie positive, ce qui limite fortement l'atténuation du gradient.

- La ReLU introduit toutefois un nouvel écueil, celui des neurones morts (dead neurons), qui restent bloqués dans leur partie négative et cessent d'apprendre. La Leaky ReLU ($f(x) = \max(\varepsilon x, x)$) corrige ce défaut en laissant passer un faible gradient même côté négatif.
- Des fonctions d'activation plus récentes, comme Swish et SwiGLU (utilisée dans plusieurs grands modèles de langage actuels), affinent encore ce principe et contribuent aux performances des architectures modernes.

Détails, étapes du greedy layer-wise pretraining et comparaison des fonctions d'activation : voir Annexe A.6.

Partie II — Géométrie des représentations neuronales

La seconde intervention de la journée change de perspective : plutôt que d'étudier comment un réseau de neurones apprend, elle interroge la nature de ce qu'il apprend, à travers une lecture géométrique des représentations internes.

Chapitre 7. Une lecture géométrique des réseaux de neurones

Chaque couche d'un réseau de neurones peut être vue comme une représentation (ou embedding) des données : elle associe à chaque exemple un point dans un espace vectoriel. Les données brutes elles-mêmes constituent déjà un embedding — une image couleur de 256 × 256 pixels est un point dans un espace à 196 608 dimensions.

Idées clés

- Le réseau ne fait, couche après couche, que déplacer un nuage de points (point cloud) dans de nouveaux espaces, jusqu'à ce que les catégories deviennent facilement séparables.
- Classifier revient ainsi, géométriquement, à rapprocher les points d'une même classe et à éloigner ceux de classes différentes — puis, en sortie, à envoyer chaque nuage vers un sommet du simplex (l'espace des distributions de probabilité).
- Chaque type de couche dispose d'un répertoire d'actions limité : une couche linéaire ne peut effectuer que des transformations affines (rotation, étirement, translation) ; la fonction ReLU agit point par point en écrasant les valeurs négatives ; la fonction softmax convertit des scores bruts (logits) en probabilités.
- C'est l'empilement de ces opérations élémentaires, simples individuellement, qui permet au réseau de réaliser des réorganisations complexes du nuage de données.

Détails, notion de simplex et description de la visualisation animée de l'entraînement par SGD : voir Annexe B.1.

Chapitre 8. Comparer deux représentations : les métriques de similarité

Cette lecture géométrique ouvre une question de recherche active en interprétabilité : deux réseaux de neurones entraînés séparément apprennent-ils la « même » représentation des données ? Plusieurs familles de métriques permettent d'apporter une réponse nuancée.

Trois grandes familles de métriques

- Les métriques par régression : on cherche à prédire les embeddings d'un réseau à partir de ceux d'un autre (typiquement par régression linéaire). Une faible erreur résiduelle indique une forte similarité. Cette approche est intuitive mais peut être trompeuse : une transformation trop permissive peut faire apparaître deux représentations comme identiques alors que l'une a perdu de l'information.
- Les métriques par alignement de noyaux (kernel alignment), dont la plus connue est le CKA (Centered Kernel Alignment) : plutôt que de comparer les coordonnées des points, on compare les relations de proximité entre les objets (distances, similarités). Cette approche reste valable même lorsque les deux réseaux produisent des embeddings de dimensions différentes, et elle est insensible aux rotations, translations et changements d'échelle.
- Les métriques par plus proches voisins (Mutual k-Nearest Neighbors, MKNN) : on vérifie si les deux réseaux s'accordent sur l'identité des voisins les plus proches de chaque point, sans exiger l'égalité exacte des distances.

Le message central de ce chapitre est qu'il n'est pas nécessaire que deux réseaux produisent des vecteurs numériquement identiques pour que l'on puisse parler de représentations équivalentes : ce qui importe est la préservation des relations structurelles entre les objets représentés.

Détails, exemples chiffrés et tableau comparatif complet des métriques : voir Annexe B.2.

Chapitre 9. L'hypothèse platonicienne de la représentation

Prolongeant les chapitres précédents, l'hypothèse platonicienne de la représentation (Platonic Representation Hypothesis) avance que des modèles d'intelligence artificielle entraînés indépendamment — voire des cerveaux biologiques — tendraient à converger vers une organisation similaire de l'information, reflétant une structure sous-jacente commune du monde.

Idées clés

- L'égalité numérique exacte des représentations n'est ni attendue ni nécessaire : les réseaux de neurones possèdent des symétries internes (une rotation globale des embeddings, compensée par une rotation équivalente du classifieur final, laisse les prédictions inchangées).
- Mesurée par la métrique MKNN, l'alignement entre modèles de vision et modèles de langage s'est renforcé au fil des générations de modèles, suggérant une convergence vers une organisation conceptuelle comparable.
- Cette tendance est toutefois moins nette lorsqu'elle est mesurée par CKA, qui peut artificiellement augmenter avec la simple augmentation de la taille des modèles, sans réelle convergence sous-jacente.
- L'accord entre modèles est plus net à un niveau sémantique global (grandes catégories) qu'à un niveau très fin (voisin le plus proche parmi un million d'exemples), où des différences d'organisation subsistent.
- Pour certaines familles de modèles, la convergence peut être démontrée mathématiquement : les apprentissages contrastifs de type InfoNCE convergent vers un noyau lié à l'information mutuelle ponctuelle ; deux modèles de type CLIP entraînés indépendamment convergent vers des embeddings reliés par une simple transformation orthogonale.

Détails sur l'alignement entre langues et sur les nuances apportées par les travaux évoqués en séance : voir Annexe B.3.

Chapitre 10. Applications et discussion finale

L'intérêt de mesurer l'alignement entre représentations n'est pas seulement théorique : il conditionne la capacité à faire coopérer des modèles entraînés séparément.

Applications évoquées

- CLIP illustre un alignement construit explicitement : un encodeur d'images et un encodeur de texte sont entraînés conjointement pour que l'embedding d'une image et celui de sa légende pointent dans la même direction.
- REPA (Representation Alignment) illustre un alignement guidé : un modèle de diffusion est entraîné pour que ses représentations internes se rapprochent de celles d'un encodeur de vision déjà pré-entraîné, ce qui accélère et stabilise son apprentissage.
- L'alignement non apparié (unpaired alignment) montre que, si deux espaces d'embeddings sont reliés par une isométrie (une transformation qui préserve les distances), la correspondance entre eux peut

être retrouvée à partir de la seule géométrie des deux nuages de points, sans exemples appariés (image-légende, phrase-traduction).

Bénéfices et limites évoqués en discussion

- Bénéfices : connexion de modèles par une simple « couche de raccordement », transfert d'apprentissage, partage de supervision entre modalités, réutilisation de représentations existantes comme fondation pour de nouveaux modèles.
- Limite potentielle : une convergence excessive entre modèles pourrait réduire la diversité des façons de raisonner (risque de « mode collapse » ou d'uniformisation des représentations).
- Question ouverte sur les world models : l'hypothèse d'une convergence géométrique entre modèles n'a pas encore été étudiée systématiquement pour les modèles du monde, mais elle serait cohérente avec l'idée que les grands modèles de langage construisent une forme de modèle interne du monde.
- La séance s'est achevée sur une question restée sans réponse, portant sur les architectures multimodales sans encodeur de vision séparé (de type Gemma), la transcription s'interrompant avant la réponse de l'intervenant.

Détails et compte rendu complet des échanges avec le public : voir Annexe B.4.

Conclusion générale

Les deux interventions de cette première journée, bien que traitant de sujets distincts, dessinent une même trajectoire intellectuelle : celle du passage de modèles simples et contraints (régression logistique, perceptron) vers des architectures profondes capables d'apprendre automatiquement des représentations complexes, puis vers une réflexion sur la nature même de ces représentations.

Fils conducteurs de la journée

- Le passage de l'ingénierie manuelle de caractéristiques à leur apprentissage automatique constitue le socle conceptuel du deep learning.
- La profondeur d'un réseau (plusieurs couches) répond à une limite pratique de l'apprentissage — et non de l'expressivité théorique — des réseaux peu profonds.
- Les difficultés d'entraînement des réseaux profonds (gradient qui s'annule) ont retardé, mais non invalidé, cette approche ; leur résolution progressive (nouvelles activations, pré-entraînement, outils de calcul automatique des gradients) a permis l'essor du deep learning moderne.
- Une fois entraînés, ces réseaux peuvent être analysés comme des systèmes géométriques qui réorganisent progressivement les données, ce qui ouvre la voie à la comparaison de modèles entre eux et à l'hypothèse d'une convergence vers des représentations du monde partagées.

Ce document restitue les notions abordées dans un cadre pédagogique et objectif ; il ne prétend pas à l'exhaustivité de l'intervention orale, dont certains échanges avec le public n'ont pu être que partiellement restitués.

Annexe — Approfondissements par chapitre

Cette annexe reprend, chapitre par chapitre, les développements, exemples et éléments visuels qui étayent les synthèses présentées dans le corps du document.

A.1 — Le problème XOR (complément du chapitre 1)

La fonction logique XOR (« ou exclusif ») retourne 0 lorsque ses deux entrées sont identiques, et 1 lorsqu'elles diffèrent :

X_1	X_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Si l'on place ces quatre points dans le plan (X_1, X_2), les points de classe 1 — (0,1) et (1,0) — se trouvent sur une diagonale, tandis que les points de classe 0 — (0,0) et (1,1) — se trouvent sur l'autre diagonale. Aucune droite ne peut séparer ces deux groupes : quel que soit son emplacement, au moins un point se retrouve du mauvais côté. Une frontière de décision valide devrait ressembler à deux régions disjointes entourant chaque paire de points de même classe — ce qui n'est plus une frontière linéaire, mais une frontière composée de plusieurs segments.

Minsky et Papert ont démontré ce résultat de façon rigoureuse en 1969, ce qui a été interprété, à tort, comme une limite générale des approches fondées sur l'apprentissage par descente de gradient — freinant la recherche sur les réseaux de neurones pendant près d'une décennie, jusqu'à la généralisation de l'usage des couches cachées et, plus tard, de la rétropropagation.

A.2 — Du feature engineering au representation learning (complément du chapitre 2)

Pour rendre le problème XOR linéairement séparable, il suffit de construire deux nouvelles variables $\phi_1(x)$ et $\phi_2(x)$ à partir des entrées originales (par exemple des combinaisons logiques telles que « x_1 ET NON x_2 »), puis d'appliquer une régression logistique standard sur ces nouvelles variables. Le classifieur reste inchangé ; seule la représentation des données change.

Cette approche, appelée feature engineering, exige une expertise humaine du domaine. En reconnaissance faciale par exemple, un ingénieur pouvait construire manuellement des variables telles que la distance entre les yeux, la largeur du nez ou la forme des sourcils. Cette méthode devient impraticable dès que le nombre de variables augmente : pour une image de plusieurs milliers de pixels, concevoir manuellement des caractéristiques pertinentes est hors de portée.

Le pipeline classique des années 1980-1990 était : Image → Feature engineering (humain) → Classifieur linéaire ou SVM → Prédiction. Le representation learning inverse cette logique : Données → Apprentissage automatique des représentations → Classifieur → Prédiction. Architecturalement, cela revient à insérer une couche cachée entre l'entrée et la sortie ; les poids de cette couche (souvent notés v_{11}, v_{12}, v_{21}, v_{22}…) sont appris par descente de gradient, au même titre que les poids du classifieur final. On parle de couche « cachée » non pas parce qu'elle est dissimulée, mais parce qu'aucune signification n'est imposée a priori à ce qu'elle apprend : seul compte le résultat final de la prédiction.

A.3 — Fonctions d'activation et couche de sortie (complément du chapitre 3)

Les deux fonctions d'activation historiquement utilisées pour les couches cachées sont :

- Sigmoïde : $\sigma(a) = 1 / (1 + e^{-a})$, à valeurs dans l'intervalle (0, 1).
- Tangente hyperbolique (tanh) : à valeurs dans l'intervalle (−1, 1), centrée sur zéro, ce qui facilite souvent l'apprentissage par rapport à la sigmoïde.

Le choix de la fonction d'activation de sortie dépend de la nature de la tâche :

Tâche	Activation de sortie	Remarque
Régression	Identité	Aucune contrainte sur la valeur prédite
Classification binaire	Sigmoïde	Renvoie une probabilité entre 0 et 1
Classification multiclasse	Softmax	Probabilités sur toutes les classes, dont la somme vaut 1

Remarque terminologique : l'intervenant emploie parfois le terme « multi-label » pour désigner ce qui correspond en réalité à la classification multiclasse (une seule classe correcte parmi plusieurs, via softmax). Une véritable classification multi-label — où plusieurs classes peuvent être vraies simultanément — repose en général sur une sortie sigmoïde indépendante pour chaque classe.

A.4 — Rétropropagation et outils de calcul (complément du chapitre 4)

L'entraînement suit le cycle : prédiction → calcul de l'erreur → calcul des gradients (règle de la chaîne) → mise à jour des poids → nouvelle prédiction. Faire ce calcul « à la main », terme par terme, serait extrêmement coûteux pour un réseau à plusieurs couches ; la rétropropagation résout ce problème en réutilisant efficacement les calculs intermédiaires, selon un principe de programmation dynamique.

L'algorithme a été redécouvert à plusieurs reprises dans l'histoire de la discipline. Deux jalons sont généralement cités : la thèse de doctorat de Paul Werbos (1974), puis l'article de David Rumelhart, Geoffrey Hinton et Ronald Williams (1986), qui en a popularisé l'usage pour l'entraînement des réseaux de neurones.

Les bibliothèques de différentiation automatique ont, elles aussi, une histoire longue : Torch (Collobert et al., 2002), Theano (issu du groupe de Yoshua Bengio, 2007-2008), Caffe, puis TensorFlow, PyTorch et plus récemment JAX. Ces outils calculent aujourd'hui automatiquement les gradients — une opération qu'il fallait auparavant implémenter manuellement — ce qui a grandement facilité l'expérimentation à grande échelle en deep learning.

A.5 — Expressivité, apprentissage et profondeur (complément du chapitre 5)

Le théorème d'approximation universelle (Hornik, 1991) porte sur l'expressivité d'un réseau à une seule couche cachée : il garantit l'existence d'une configuration de poids permettant d'approximer n'importe quelle fonction continue, sans garantir que cette configuration soit atteignable par un algorithme d'apprentissage donné. Une illustration intuitive : la combinaison de plusieurs fonctions en forme de « marche » ou de plusieurs sigmoïdes permet déjà de construire des frontières de décision courbes, puis, en en combinant davantage, des frontières arbitrairement complexes (par exemple des régions imbriquées les unes dans les autres).

Le recours à des réseaux profonds (plusieurs couches successives de projections non linéaires) répond à une intuition simple : au lieu de tenter de séparer des données mêlées en une seule transformation potentiellement très large, chaque couche effectue une séparation partielle, progressive. Illustration schématique :

- Données initiales : classes mélangées (○ ● ○ ● ● ○)
- Après la 1re couche : début de regroupement (○ ○ ● ● ○ ●)
- Après la 2e couche : regroupement plus marqué (○ ○ ○ ● ● ●)
- Après la 3e couche : classes presque parfaitement séparables, qu'une simple régression logistique finale peut alors distinguer.

Ce processus est entièrement piloté par les données : aucune consigne explicite n'indique au réseau selon quel critère séparer à chaque étape ; ces représentations intermédiaires émergent de la descente de gradient. C'est précisément ce mécanisme qui n'a pas fonctionné comme attendu dans les années 1990 (voir chapitre 6 et annexe A.6), en raison du problème du gradient qui s'annule.

A.6 — Le gradient qui s'annule : mécanisme et remèdes (complément du chapitre 6)

Mécanisme : la dérivée de la sigmoïde est toujours inférieure à 0,25, et proche de zéro lorsque les poids sont initialisés près de zéro. Lorsque le gradient de l'erreur est propagé de la sortie vers les premières couches, il est multiplié, à chaque couche traversée, par un facteur de cet ordre. Pour un réseau à cinq couches ou plus, ce produit de facteurs très inférieurs à 1 fait tendre le gradient vers zéro dans les couches les plus profondes, qui cessent alors d'apprendre efficacement.

Étapes du greedy layer-wise pretraining (Hinton et al., 2006 ; étendu par Bengio et al. aux autoencodeurs) :

- 1. Entraîner un autoencodeur reliant les données X à une représentation H1, puis à une reconstruction de X ; ne conserver que l'encodeur (X → H1).
- 2. Figer cette première couche, puis entraîner un second autoencodeur reliant H1 à H2, et ainsi de suite pour chaque couche supplémentaire.
- 3. Empiler les encodeurs ainsi obtenus (X → H1 → H2 → … → Hk), ajouter une couche de classification, puis affiner l'ensemble du réseau sur la tâche finale (supervised fine-tuning).

Il est important de noter que cette méthode n'élimine pas mathématiquement le problème du gradient qui s'annule : l'architecture du réseau reste inchangée, et le gradient continue de s'atténuer pendant la phase de fine-tuning. Le bénéfice est indirect : les couches disposent déjà, grâce au pré-entraînement, de représentations utiles, ce qui rend l'entraînement supervisé plus stable en pratique — un principe qui préfigure le couple pré-entraînement / fine-tuning des grands modèles de langage actuels.

La solution la plus directe consiste à remplacer les fonctions d'activation saturantes (sigmoïde, tanh) par la ReLU, f(x) = max(0, x), dont la dérivée vaut exactement 1 sur la partie positive — le gradient traverse alors les couches sans être atténué lorsque le neurone est actif. Le revers de cette solution est le phénomène des neurones morts : un neurone dont l'entrée reste toujours négative a une sortie et un gradient constamment nuls, et cesse donc d'apprendre. La Leaky ReLU, f(x) = max(εx, x) avec ε petit (par exemple 0,01), corrige ce défaut en laissant passer un gradient résiduel côté négatif. Des activations plus récentes, telles que Swish (x·σ(x)) et SwiGLU — qui combine une activation de type Swish à un mécanisme de porte (gating) contrôlant l'information transmise —, sont aujourd'hui utilisées dans de nombreux modèles de grande taille.

B.1 — Embeddings, nuages de points et simplex (complément du chapitre 7)

Un embedding associe à chaque exemple (une image, un mot…) un vecteur de nombres, c'est-à-dire un point dans un espace de grande dimension. Deux exemples sémantiquement proches (deux images de voitures, par exemple) tendent à être associés à des points proches dans cet espace, tandis que deux exemples très différents (une voiture et un requin-baleine) en sont éloignés — un phénomène qui apparaît de manière constante quel que soit l'objectif d'entraînement (classification supervisée, apprentissage auto-supervisé, apprentissage contrastif).

Chaque couche du réseau produit son propre nuage de points, obtenu en réorganisant celui de la couche précédente. Les données brutes elles-mêmes constituent le tout premier de ces nuages : une image couleur de 256 × 256 pixels est un point dans un espace à 256 × 256 × 3 = 196 608 dimensions.

Le simplex est l'espace de toutes les distributions de probabilité possibles sur un ensemble de classes. Pour une classification binaire, il s'agit du segment reliant les points (1, 0) et (0, 1) : un point (0,5 ; 0,5) sur ce segment correspond à une incertitude totale entre les deux classes. Pour trois classes, le simplex devient un triangle, dont chaque sommet représente une certitude absolue en faveur d'une classe. L'objectif de l'entraînement peut alors se lire géométriquement : déplacer chaque point du nuage de données vers le sommet du simplex correspondant à sa vraie classe.

Chaque type de couche ne dispose que d'un répertoire d'opérations restreint :

- Une couche linéaire ne peut réaliser que des transformations affines du nuage de points : rotation, étirement, compression, translation — jamais de courbure.
- La fonction ReLU agit indépendamment sur chaque coordonnée : elle « écrase » sur zéro toute valeur négative et laisse inchangée toute valeur positive, introduisant ainsi la non-linéarité nécessaire.
- La fonction softmax convertit des scores bruts (logits), non contraints, en un point du simplex — c'est-à-dire en une distribution de probabilité valide.

La démonstration présentée en séance faisait visualiser, pour un petit réseau de largeur 2 entraîné par descente de gradient stochastique (SGD), comment l'alternance de couches linéaires et de ReLU étire, replie puis sépare progressivement deux nuages de points (par exemple deux classes « chat » et « chien »), jusqu'à ce que softmax les projette chacun vers un sommet distinct du simplex.

B.2 — Métriques de similarité représentationnelle (complément du chapitre 8)

Formellement, une métrique de similarité représentationnelle est une fonction $S(f_1, f_2)$ qui prend en entrée deux réseaux (ou deux couches) et renvoie un score : proche de 1 pour des représentations quasi identiques, proche de 0 pour des représentations très différentes. En pratique, on ne compare pas directement les réseaux mais l'action de chacun sur un même ensemble de points de test (probe points), ce qui produit deux nuages d'embeddings Z1 et Z2 à comparer.

Métriques par régression

On mesure l'erreur résiduelle d'une régression (le plus souvent linéaire) entraînée à prédire les embeddings du second réseau à partir de ceux du premier. Deux nuages reliés par une simple rotation obtiennent ainsi un score de similarité maximal, ce qui est cohérent avec l'intuition que seule la structure relative importe, non la position absolue des vecteurs.

Cette approche présente une faiblesse : une transformation trop permissive peut masquer une perte d'information. Par exemple, un réseau qui ignore totalement l'une des deux dimensions d'entrée (en renvoyant systématiquement 0 pour cette dimension) peut malgré tout obtenir un score de similarité parfait avec un réseau qui, lui, conserve cette information — la relation « garder Y, envoyer X vers 0 » étant elle-même une transformation linéaire valide. Des correctifs existent : exiger une matrice de transformation inversible, symétriser la mesure en régressant dans les deux sens, ou autoriser une régression non linéaire (par exemple un petit réseau) pour tolérer un écart plus large entre les deux représentations.

Métriques par alignement de noyaux (kernel alignment) et CKA

Plutôt que de comparer les coordonnées, cette famille de métriques compare les relations de proximité entre les objets représentés : la matrice de distances (ou de similarités) entre toutes les paires d'exemples, appelée noyau (kernel). Deux représentations peuvent ainsi être jugées équivalentes même si leurs dimensions numériques diffèrent — un embedding de dimension 128 peut être comparé à un embedding de dimension 2048, car seules les distances entre exemples sont mises en correspondance.

Le CKA (Centered Kernel Alignment) est la métrique la plus utilisée de cette famille. Le terme « centré » indique que la moyenne des données est retirée avant comparaison, à la manière du centrage préalable au calcul d'une corrélation en statistiques. Le CKA considère comme équivalentes deux représentations reliées par une rotation, une translation, une symétrie ou un changement d'échelle — ces transformations, appelées isométries lorsqu'elles préservent les distances, sont traitées comme des détails techniques sans importance (nuisances).

Métriques par plus proches voisins (MKNN)

Plutôt que de comparer des distances exactes, ces métriques vérifient uniquement si les deux réseaux s'accordent sur l'identité des voisins les plus proches de chaque point (une mesure de nature ordinale plutôt que continue). Le Mutual k-Nearest Neighbors (MKNN) calcule, pour chaque point, la proportion de voisins communs entre les deux représentations.

Métrique	**Principe**	**Avantage**	**Limite**
Régression	Peut-on prédire les embeddings d'un réseau à partir de l'autre ?	Simple, intuitive	Peut masquer une perte d'information si la transformation autorisée est trop permissive
Kernel / CKA	Les deux réseaux préservent-ils les mêmes relations de proximité ?	Fonctionne même à dimensions différentes ; robuste aux rotations et changements d'échelle	Peut artificiellement augmenter avec la seule taille du modèle
MKNN	Les deux réseaux s'accordent-ils sur les plus proches voisins ?	Mesure locale, robuste, de nature ordinale	Ne renseigne pas sur la structure globale de l'espace

Au-delà de ces trois familles, l'intervenant a évoqué des approches fondées sur l'information mutuelle (deux représentations véhiculent-elles la même information, au sens formel du terme ?) et sur la cohérence cyclique (cycle consistency), qui teste l'existence d'une fonction bijective reliant les deux espaces, sans contrainte de linéarité. Interrogé sur sa préférence, l'intervenant a indiqué privilégier les métriques de type noyau, en avançant que deux représentations mesurant les distances de la même façon se comportent de manière équivalente pour la plupart des usages pratiques (recherche des plus proches voisins, régression

linéaire, régime des noyaux tangents neuronaux), même si cette position reste, de son propre aveu, un point de vue théorique et non une conclusion démontrée dans l'absolu.

B.3 — Hypothèse platonicienne : compléments (complément du chapitre 9)

Une démonstration interactive proposée au public a consisté à comparer un jugement de similarité visuelle humain à celui d'un modèle de vision (DINO) : face à trois images, l'assistance devait indiquer laquelle des deux images latérales était la plus proche de l'image centrale. Le consensus du public a coïncidé avec le jugement du modèle, illustrant que les réseaux de neurones et les humains semblent souvent mesurer la similarité visuelle de façon comparable.

L'alignement entre modèles de vision et modèles de langage, mesuré par MKNN, s'est renforcé au fil des générations de modèles jusque vers 2024 environ ; au-delà, la tendance devient plus difficile à étudier car les modèles de langage open source ont cessé de progresser significativement en perplexité de prédiction du mot suivant, la recherche s'étant réorientée vers le post-entraînement (raisonnement, code, mathématiques) plutôt que vers l'amélioration des représentations de base.

Interrogé sur une possible baisse récente d'alignement entre modèles récents, l'intervenant a avancé trois explications possibles : l'incertitude des résultats sur l'alignement avec le cerveau biologique (domaine jugé encore mal stabilisé) ; l'effet du post-entraînement, qui modifierait la nature des embeddings extraits des modèles de langage actuels par rapport aux modèles simplement pré-entraînés ; et, de façon plus spéculative, l'effet de contenus synthétiques générés par IA sur le web, jugé possible mais peu probable comme explication principale à ce stade.

Concernant l'alignement entre langues humaines, l'intervenant a indiqué que la similarité géométrique entre langues (par exemple entre l'anglais et le français) est nettement plus forte que celle observée entre modalités (vision et langage). Si certains concepts culturellement spécifiques ne se traduisent pas parfaitement d'une langue à l'autre, la plupart des concepts courants (pomme, personne, amour, haine…) existent dans l'ensemble des langues, ce qui explique un socle géométrique largement partagé.

Un point de nuance a également été apporté par des travaux tiers cités en séance : la convergence observée serait avant tout locale et de nature ordinale (préservation de l'ordre relatif des distances) plutôt qu'une convergence stricte vers une même géométrie globale ; le CKA global montre une tendance beaucoup plus faible que le MKNN local, ce qui a pu conduire certains commentateurs à conclure, de façon excessive, à une absence totale de convergence structurelle entre modèles.

B.4 — Applications et échanges avec le public (complément du chapitre 10)

Des résultats théoriques, et non seulement empiriques, viennent étayer certaines de ces observations pour des familles de modèles spécifiques : pour l'apprentissage contrastif de type InfoNCE, il peut être démontré que le noyau appris converge vers une fonction liée à l'information mutuelle ponctuelle (pointwise mutual information) ; pour des modèles de type CLIP entraînés indépendamment, il peut être démontré que leurs embeddings convergent vers une correspondance reliée par une même transformation orthogonale, appliquée de façon identique aux embeddings visuels et textuels.

Sur le plan applicatif, l'alignement des représentations permet notamment : de connecter des modèles de modalités différentes par une simple couche de raccordement (stitching layer) plutôt que par un

réentraînement complet ; de transférer un apprentissage d'un modèle à un autre ; de partager de la supervision entre modalités ; et de servir de fondation à de nouveaux modèles par affinage (fine-tuning).

L'intervenant a néanmoins souligné un risque potentiel : une convergence excessive entre modèles pourrait réduire la diversité des façons de raisonner, un phénomène rapproché du « mode collapse », où l'ensemble des systèmes d'IA finiraient par partager une seule et même façon de représenter le monde.

Deux questions du public sont restées partiellement ou totalement sans réponse dans la transcription disponible : l'une portant sur l'extension de cette hypothèse aux world models (modèles du monde), à laquelle l'intervenant a répondu que la question restait largement ouverte mais plausible au regard du débat actuel sur la nature de modèle du monde des grands modèles de langage ; l'autre, posée en toute fin de séance, portant sur les architectures multimodales sans encodeur de vision séparé (de type Gemma), pour laquelle la transcription s'interrompt avant que l'intervenant n'ait pu répondre, la session s'étant achevée sur une pause déjeuner.